

Live Aircraft & Nearby Airports Explorer

1. Introduction
2. Scope
3. Project Structure
4. Task distribution
5. Initial API list and backend layer plan

Introduction

Group members: Ahmed, Brooklyn, Hamza

We want to create a web application called “Live Aircraft & Nearby Airports Explorer”.

The purpose of this project is to let users explore real-time aviation activity around any location.

The system will:

- Let the user search for a city or place (geocoding)
- Show nearby airports (from OurAirports data)
- Show live aircraft positions in that area (OpenSky Network API)
- Display everything on an interactive Folium map (airport markers + aircraft markers with call sign, altitude, speed)
- Save search history and aircraft snapshots into an SQLite database
- Provide a control panel to view previous searches, compare traffic between different times, and manage saved airports
- Generate simple charts (e.g., number of aircraft per hour, busiest airports in saved area)

The user will receive:

- A map with airports and live aircraft
- A table/list of aircraft details
- Historical data that persists after server restart
- Charts showing traffic patterns

Scope

Core features:

- Create SQLite database with tables: searches, airports, aircraft_snapshots, saved_airports
- Build a Flask web application (routes, templates, Bootstrap)

Implement API clients:

- Open-Meteo Geocoding API – convert city name to coordinates
- OurAirports data (static CSV or pre-loaded into DB) – airport name, ICAO/IATA, coordinates
- OpenSky Network API – live aircraft states within a bounding box

Implement routine:

- Create routine to fetch live aircraft data every X seconds (or on user refresh)
- Create routine to find nearby airports (distance calculation)
- Create Folium map with two marker layers (airports + aircraft)

Create control panel for:

- View saved search history
- View saved aircraft snapshots with timestamps
- Compare current traffic vs previous snapshot
- Delete or bookmark an airport

Generate at least one meaningful chart (e.g. aircraft count per hour using Plotly or Matplotlib)

Error validation (API failures, no aircraft, invalid location) – handled together

Project Structure

```
/aircraft_explorer
├── app.py           # Flask main
├── db.py           # SQLite helpers
├── apis.py         # Geocoding, OpenSky, OurAirports loader
├── map_utils.py    # Folium map generation
├── charts.py       # Plotly/Matplotlib charts
├── templates/
│   ├── index.html
│   ├── history.html
│   └── control.html
├── static/
│   └── style.css
├── data/
│   └── airports.csv # OurAirports dataset (pre-download)
└── README.md
```

Task distribution

Task	Responsibility	Effort
Creating SQLite database (tables, indexes)	Brooklyn	Low
Finding & evaluating APIs	Brooklyn	Low
API function for geocoding (city → coordinates)	Hamza	Low
API function for live aircraft (OpenSky)	Ahmed	Medium
Loading OurAirports data into DB	Ahmed	Medium
Function to find nearby airports (distance calc)	Hamza	Medium
Creating Folium map with markers	Brooklyn	High
Creating Flask routes & templates (UI)	Hamza	High
Control panel logic (save/delete/compare)	Ahmed	Medium
Charts (aircraft count over time)	Brooklyn	Medium
Error validation & final integration	Together	High

Initial API list & backend plan

Open-Meteo Geocoding: <https://open-meteo.com/en/docs/geocoding-api>

OpenSky Network API: <https://openskynetwork.github.io/opensky-api/rest.html>

OurAirports data: <https://ourairports.com/data/>